

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: Operating Systems

COURSE CODE: CSA0403

COURSE OUTCOME COVERED

C03: Evaluate memory management mechanisms such as linking, paging, and segmentation to determine their effectiveness for different system requirements

Assessment Tool Weightage: 40%

QUESTIONS: 5

Total Marks: 50

High Experiments

Code of Conduct:

I **ANISH K/192521288** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



Experiment Questions:

- 1.Design and implement a program to simulate the translation of a logical address into a physical address using base and limit registers. The program should accept logical address as input, check for validity, and compute the corresponding physical address. Display appropriate messages for valid and invalid addresses.
- 2.Write a program to demonstrate the paging memory management scheme. The program should divide memory into fixed-size pages and frames, accept page number and offset as input, and compute the corresponding physical address using a page table.
- 3.Develop a program to simulate a page table. The program should allow the user to input page table entries and logical addresses, then map them to physical addresses. Clearly show how page numbers are translated into frame numbers.
- 4.Implement a program to simulate the working of a Translation Lookaside Buffer (TLB). The program should demonstrate how TLB improves memory access time by calculating the effective memory access time with and without TLB.
- 5.Write a program to illustrate memory protection using base and limit registers. The program should verify whether a given logical address lies within the valid range and prevent unauthorized access.
6. Develop a program to simulate dynamic storage allocation using First Fit, Best Fit, and Worst Fit strategies. The program should accept the sizes of available memory blocks and process requests, allocate memory using each strategy, and display the allocated blocks and remaining unused memory.
7. Write a program to simulate swapping between main memory and secondary storage. The program should accept the memory size, process sizes, and process arrival/removal requests, and display which processes are swapped in and swapped out at each stage.
8. Implement a program to simulate segmentation memory management. The program should accept segment numbers and offsets along with a segment table containing base and limit values. It should validate the offset and calculate the corresponding physical address for valid memory references.
9. Develop a program to demonstrate segmentation with paging. The program should accept a logical address consisting of segment number, page number, and offset, use the segment table and page table for translation, and display the final physical address or an appropriate error message.
10. Write a program to simulate virtual memory using demand paging. The program should accept a sequence of page references and the number of available frames, identify page faults, load required pages into memory, and display the contents of each frame after every reference.
11. Implement a program to compare FIFO, LRU, and Optimal page replacement algorithms. The program should accept a page-reference string and number of available frames, simulate all three algorithms, display the frame contents after each reference, and calculate the total number of page faults for each algorithm.

12. Develop a program to simulate thrashing and working-set management. The program should accept a page-reference sequence and working-set window size, determine the pages currently required by a process, calculate the working-set size at different intervals, and identify situations where excessive page faults may cause thrashing.

13. Write a program to simulate dynamic linking. The program should maintain a symbol table containing functions from dynamically linked libraries and allow a program to request a function at execution time. The program should determine whether the required symbol is available and display the corresponding library and memory address.

14. Implement a program to simulate a linker using a simplified two-pass approach. The program should accept multiple object modules containing symbols and references, construct an external symbol table during Pass 1, resolve external references during Pass 2, and display the final linked memory addresses.

15. Develop a program to simulate disk scheduling algorithms such as FCFS, SSTF, SCAN, and C-SCAN. The program should accept the initial disk-head position and a sequence of disk requests, calculate the head movement for each algorithm, display the order in which requests are serviced, and determine the total head movement.

RUBRICS FOR CALCULATION:

Criteria	Weightage (Marks)	Level 4 - Exemplary (Full Marks)	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning
Conceptual Understanding	35 Marks	Demonstrates clear, complete, and in-depth understanding of OS concepts (types, structure, system calls)	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor or incorrect understanding
Accuracy of Answers	25 Marks	All answers are correct, precise, and relevant	Mostly correct with minor errors	Partially correct with noticeable errors	Mostly incorrect or irrelevant
Application of Knowledge	15 Marks	Effectively applies concepts to scenarios/questions	Applies with minor mistakes	Limited or partial application	No meaningful application
Completeness of Response	15 Marks	All parts of each question are fully answered	Most parts covered with small omissions	Some parts missing	Incomplete answers
Clarity & Presentation	10 Marks	Clear, well-structured, and neatly presented answers	Understandable with minor issues	Somewhat unclear or poorly structured	Difficult to understand

01. Translation of Logical Address to Physical Address Using Base and Limit Registers

PSEUDO CODE:

```
Start

Input Base Register value
Input Limit Register value
Input Logical Address

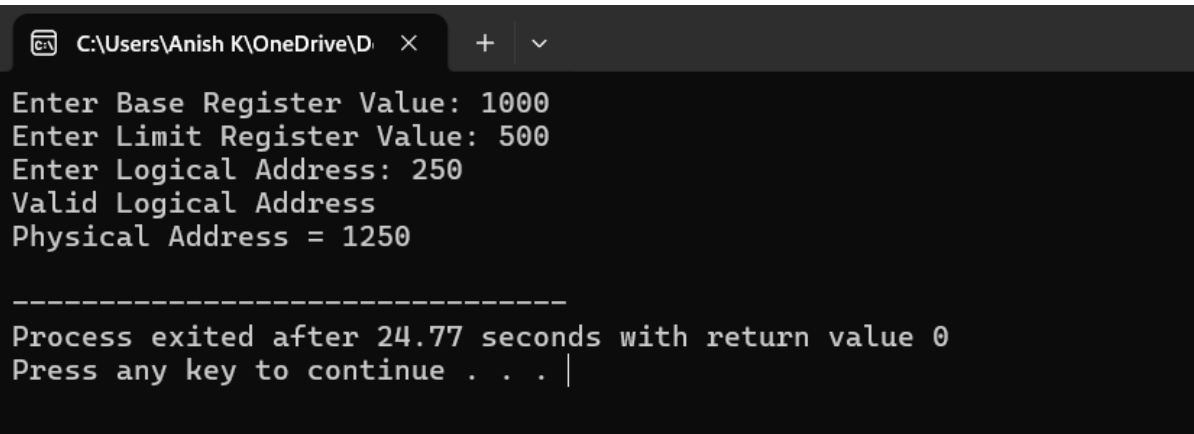
If Logical Address >= 0 AND Logical Address < Limit Register
    Physical Address = Base Register + Logical Address
    Display "Valid Logical Address"
    Display Physical Address
Else
    Display "Invalid Logical Address"

Stop
```

Code Explanation :

- The program begins by accepting the Base Register, Limit Register, and Logical Address from the user.
- The Base Register stores the starting address of the process in physical memory.
- The Limit Register stores the maximum size of the process (memory limit).
- The program checks whether the logical address is valid by verifying that it is greater than or equal to 0 and less than the Limit Register.
- If the logical address is valid, the physical address is calculated using the formula:
$$\text{Physical Address} = \text{Base Register} + \text{Logical Address}$$
- The calculated physical address is displayed.
- If the logical address exceeds the limit, the program displays "Invalid Logical Address!" because accessing memory beyond the limit is not allowed.
- Finally, the program terminates.

OUTPUT:



```
C:\Users\Anish K\OneDrive\D  ×  +  v

Enter Base Register Value: 1000
Enter Limit Register Value: 500
Enter Logical Address: 250
Valid Logical Address
Physical Address = 1250

-----
Process exited after 24.77 seconds with return value 0
Press any key to continue . . . |
```

02. Paging Memory Management Scheme

PSEUDO CODE:

Start

Input number of pages

Input page size

For each page

Input frame number and store in page table

Input page number

Input offset

If page number is invalid

Display "Invalid Page Number"

Else if offset is invalid

Display "Invalid Offset"

Else

Find frame number from page table

Physical Address = (Frame Number × Page Size) + Offset

Display frame number

Display physical address

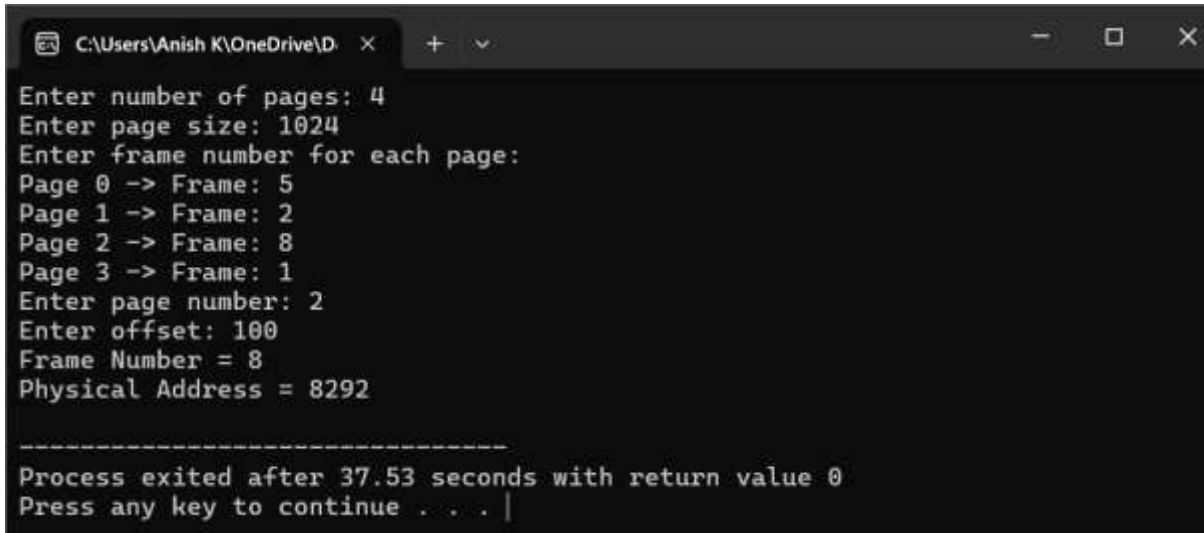
Stop

Code Explanation:

- The program accepts the number of pages and the page size.
- It creates a page table, where each page is mapped to a frame number.
- The user enters the frame number for every page.
- The program then accepts a page number and an offset.
- It checks whether the page number exists in the page table.
- It also verifies that the offset is less than the page size.
- If both values are valid, the corresponding frame number is obtained from the page table.

- The physical address is calculated using:
- $\text{Physical Address} = (\text{Frame Number} \times \text{Page Size}) + \text{Offset}$
- Finally, the program displays the frame number and the computed physical address. If the page number or offset is invalid, an appropriate error message is displayed.

OUTPUT:



```
C:\Users\Anish K\OneDrive\D >
Enter number of pages: 4
Enter page size: 1024
Enter frame number for each page:
Page 0 -> Frame: 5
Page 1 -> Frame: 2
Page 2 -> Frame: 8
Page 3 -> Frame: 1
Enter page number: 2
Enter offset: 100
Frame Number = 8
Physical Address = 8292

-----
Process exited after 37.53 seconds with return value 0
Press any key to continue . . . |
```

03. Page Table Simulation

PSEUDO CODE:

```
Start
Input number of pages
Input page size
For each page
    Input corresponding frame number
    Store it in the page table
Display the page table
Input page number
Input offset
If page number is invalid
    Display "Invalid Page Number"
Else if offset is invalid
    Display "Invalid Offset"
Else
    Find frame number from page table
    Physical Address = (Frame Number × Page Size) + Offset
    Display page number and frame number
    Display physical address
Stop
```

Code Explanation:

- The program accepts the number of pages and the page size from the user.
- It creates a page table by storing the frame number corresponding to each page.
- The complete page table is displayed so that the page-to-frame mapping is visible.
- The user enters a logical address, which consists of a page number and an offset.
- The program checks whether the page number is within the valid range.
- It also checks whether the offset is less than the page size.
- If both values are valid, the frame number is obtained from the page table.
- The program clearly shows the translation in the format Page X → Frame Y.
- The physical address is calculated using the formula:

- $\text{Physical Address} = (\text{Frame Number} \times \text{Page Size}) + \text{Offset}$
- If the page number or offset is invalid, an appropriate error message is displayed.

OUTPUT:

```

C:\Users\Anish K\OneDrive\D... X + v
Enter number of pages: 4
Enter page size: 1024
Enter frame number for each page:
Page 0 -> Frame: 5
Page 1 -> Frame: 2
Page 2 -> Frame: 7
Page 3 -> Frame: 1

Page Table:
Page No Frame No
0        5
1        2
2        7
3        1

Enter Logical Address
Page Number: 2
Offset: 300

Translation:
Page 2 --> Frame 7
Physical Address = 7468

-----
Process exited after 41.89 seconds with return value 0
Press any key to continue . . . |

```

04.Simulation of Translation Lookaside Buffer (TLB)

PSEUDO CODE:

```
Start
Input Memory Access Time
Input TLB Access Time
Input TLB Hit Ratio
Calculate Memory Access Time without TLB
    WithoutTLB = 2 × Memory Access Time
Calculate Effective Memory Access Time with TLB
    WithTLB = HitRatio × (TLB Time + Memory Time)
        + (1 – HitRatio) × (TLB Time + 2 × Memory Time)
Display Memory Access Time without TLB
Display Effective Memory Access Time with TLB
If WithTLB < WithoutTLB
    Display "TLB improves memory access time"
Else
    Display "No improvement in memory access time"
Stop
```

Code Explanation:

- The program accepts the Memory Access Time (MAT), TLB Access Time (TAT), and TLB Hit Ratio from the user.
- It calculates the memory access time without a TLB, where two memory accesses are required (one for the page table and one for the actual data).
- The program then calculates the Effective Memory Access Time (EMAT) using the TLB hit ratio.
- If there is a TLB hit, the address translation is found in the TLB, requiring only one memory access.
- If there is a TLB miss, the system accesses both the page table and the memory, resulting in two memory accesses.

- The effective memory access time is calculated using the formula:
- $EMAT = h \times (TLB\ Time + Memory\ Time) + (1 - h) \times (TLB\ Time + 2 \times Memory\ Time)$
- where h is the TLB hit ratio.
- The program compares the access times with and without the TLB.
- If the effective memory access time is lower, it displays that the TLB improves memory access time; otherwise, it indicates no improvement.

OUTPUT:

```

C:\Users\Anish K\OneDrive\D  ×  +  ∨
Enter Memory Access Time (ns): 100
Enter TLB Access Time (ns): 20
Enter TLB Hit Ratio (0 to 1): 0.8

Memory Access Time without TLB = 200.00 ns
Effective Memory Access Time with TLB = 140.00 ns
TLB improves memory access time.

-----
Process exited after 17.83 seconds with return value 0
Press any key to continue . . . |

```

05. Memory Protection Using Base and Limit Registers

PSEUDO CODE:

```
Start
Input Base Register value
Input Limit Register value
Input Logical Address

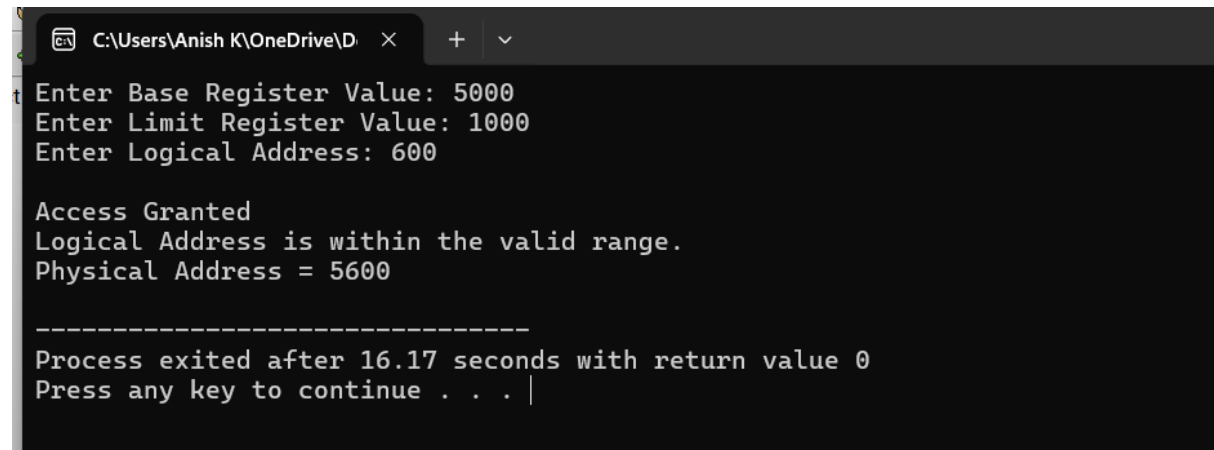
If Logical Address >= 0 AND Logical Address < Limit Register
    Physical Address = Base Register + Logical Address
    Display "Access Granted"
    Display Physical Address
Else
    Display "Access Denied"
    Display "Memory Protection Violation"
Stop
```

Code Explanation:

- The program accepts the Base Register, Limit Register, and Logical Address as input.
- The Base Register contains the starting physical address of the process.
- The Limit Register specifies the maximum memory size allocated to the process.
- The program checks whether the logical address is greater than or equal to 0 and less than the limit register value.
- If the logical address is valid, the program grants access and calculates the physical address using:
 - $\text{Physical Address} = \text{Base Register} + \text{Logical Address}$
- The physical address is then displayed.
- If the logical address is outside the valid range, the program denies access and displays a Memory Protection Violation message.

- This mechanism prevents a process from accessing memory outside its allocated region, ensuring memory protection.

OUTPUT:



```
C:\Users\Anish K\OneDrive\D >
Enter Base Register Value: 5000
Enter Limit Register Value: 1000
Enter Logical Address: 600

Access Granted
Logical Address is within the valid range.
Physical Address = 5600

-----
Process exited after 16.17 seconds with return value 0
Press any key to continue . . . |
```

06. Dynamic Storage Allocation (First Fit, Best Fit, Worst Fit)

PSEUDO CODE:

Start

Input number of memory blocks

Input size of each memory block

Input number of processes

Input size of each process

Perform First Fit allocation

Display allocated blocks

Display remaining memory

Perform Best Fit allocation

Display allocated blocks

Display remaining memory

Perform Worst Fit allocation

Display allocated blocks

Display remaining memory

Stop

Code Explanation :

- The program accepts the number of memory blocks and their sizes.
- It then accepts the number of processes and their memory requirements.
- First Fit allocates each process to the first memory block that is large enough.
- Best Fit allocates each process to the smallest memory block that can accommodate it, minimizing wasted space.
- Worst Fit allocates each process to the largest available memory block, leaving larger free blocks after allocation.

- After allocating a process, the remaining size of the selected memory block is updated.
- If no suitable block is available, the process is marked as Not Allocated.
- Finally, the program displays the allocated block for each process and the remaining unused memory after each allocation strategy.

OUTPUT:

```

C:\Users\Anish K\OneDrive\D
Enter number of memory blocks:
5
Enter sizes of memory blocks:
100
500
200
300
600
Enter number of processes: 4
Enter sizes of processes:
212
417
112
426

FIRST FIT Allocation
Process Size      Block
P1      212      2
P2      417      5
P3      112      2
P4      426      Not Allocated
Remaining Block Sizes:
Block 1 = 100
Block 2 = 176
Block 3 = 200
Block 4 = 300
Block 5 = 183

BEST FIT Allocation
Process Size      Block
P1      212      4
P2      417      2
P3      112      3
P4      426      5
Remaining Block Sizes:
Block 1 = 100
Block 2 = 83
Block 3 = 88
Block 4 = 88
Block 5 = 174

```

07. Simulation of Swapping Between Main Memory and Secondary Storage

PSEUDO CODE:

```
Start
Input main memory size
Input number of processes
Input size of each process
Repeat until exit
Display menu:
    1. Swap In
    2. Swap Out
    3. Display Memory Status
    4. Exit
If choice is Swap In
    Input process number
    Check available memory
    If memory available
        Add process size to used memory
        Display process swapped in
    Else
        Display insufficient memory
If choice is Swap Out
    Input process number
    Remove process size from used memory
    Display process swapped out

If choice is Display
    Show used and free memory
Stop
```

Code Explanation:

- The program simulates the swapping mechanism used in operating systems.
- It accepts the main memory size and the sizes of multiple processes.
- The variable usedMemory keeps track of the memory currently occupied by swapped-in processes.
- When a process is requested to swap in, the program checks whether enough memory is available.
- If sufficient space exists, the process is moved from secondary storage to main memory.
- When a process is requested to swap out, it is removed from main memory and returned to secondary storage.
- The program continuously displays memory usage and free memory after each operation.
- It prevents swapping in a process when there is insufficient memory space.

OUTPUT:

```
C:\Users\Anish K\OneDrive\D  x  +  v
200
300
400
250

Initial Memory Status

1. Swap In Process
2. Swap Out Process
3. Display Memory Status
4. Exit
Enter your choice: 1
Enter process number to swap in: 2
Process P2 swapped into main memory

1. Swap In Process
2. Swap Out Process
3. Display Memory Status
4. Exit
Enter your choice:
3

Current Memory Usage: 300 / 1000
Free Memory: 700

1. Swap In Process
2. Swap Out Process
3. Display Memory Status
4. Exit
Enter your choice: 4

-----
Process exited after 54.04 seconds with return value 0
Press any key to continue . . . |
```

08. Segmentation Memory Management Simulation

PSEUDO CODE:

```
Start
Input number of segments
For each segment
    Input base address
    Input limit value
    Store in segment table
Input segment number
Input offset
If segment number is invalid
    Display "Invalid Segment Number"
Else
    Get base and limit from segment table
    If offset < limit
        Physical Address = Base + Offset
        Display valid memory reference
        Display physical address
    Else
        Display "Invalid Offset"
        Display memory access violation
Stop
```

Code Explanation:

- The program simulates segmentation memory management used in operating systems.
- It creates a segment table where each segment contains two values:
- Base Address: Starting physical address of the segment.
- Limit: Maximum size of the segment.
- The user enters a segment number and an offset as a logical address.
- The program checks whether the given segment number exists in the segment table.
- It retrieves the corresponding base and limit values from the segment table.
- The offset is validated by comparing it with the segment limit.

- If the offset is within the valid range, the physical address is calculated using:
- $\text{Physical Address} = \text{Base Address} + \text{Offset}$
- If the offset exceeds the segment limit, the program prevents memory access and displays an error message.
- This ensures protection by preventing unauthorized access beyond segment boundaries.

OUTPUT:

```
C:\Users\Anish K\OneDrive\D  ×  +  v
Enter number of segments: 3
Enter Base and Limit values for each segment:
Segment 0 Base: 1000
Segment 0 Limit: 500
Segment 1 Base: 2000
Segment 1 Limit: 400
Segment 2 Base: 3000
Segment 2 Limit: 00

Enter Segment Number: 1
Enter Offset: 500

Invalid Offset!
Memory Access Violation

-----
Process exited after 43.7 seconds with return value 0
Press any key to continue . . . |
```

09. Segmentation with Paging

PSEUDO CODE:

```
Start
Input number of segments
Input page size
For each segment
    Input number of pages
    Store page limit
    Input frame number for each page
Input segment number
Input page number
Input offset
If segment number is invalid
    Display error
Else if page number is invalid
    Display error
Else if offset is invalid
    Display error
Else
    Get frame number from page table
     $\text{Physical Address} = (\text{Frame Number} \times \text{Page Size}) + \text{Offset}$ 
    Display Segment → Page → Frame mapping
    Display Physical Address
Stop
```

Code Explanation:

- The program accepts the number of segments and the page size.
- For each segment, the user enters the number of pages and the frame number corresponding to each page.
- The segment table stores the page limit (number of pages) for every segment.

- The user enters a logical address consisting of a segment number, page number, and offset.
- The program first checks whether the segment number is valid.
- It then checks whether the page number exists within the selected segment.
- Next, it verifies that the offset is less than the page size.
- If all values are valid, the corresponding frame number is obtained from the page table.
- The physical address is calculated using:
- $\text{Physical Address} = (\text{Frame Number} \times \text{Page Size}) + \text{Offset}$
- The program displays the complete translation in the form:
- Segment $\rightarrow$ Page $\rightarrow$ Frame $\rightarrow$ Physical Address
- If any input is invalid, an appropriate error message is displayed.

OUTPUT:

```

C:\Users\Anish K\OneDrive\D... x + v
Enter number of segments: 2
Enter page size: 1024

Enter number of pages in Segment 0: 2
Enter frame number for each page:
Page 0 -> Frame: 5
Page 1 -> Frame: 8

Enter number of pages in Segment 1: 3
Enter frame number for each page:
Page 0 -> Frame:
2
Page 1 -> Frame: 6
Page 2 -> Frame: 9

Enter Logical Address
Segment Number:
1
Page Number: 2
Offset: 100

Address Translation
Segment 1 --> Page 2 --> Frame 9
Physical Address = 9316

-----
Process exited after 36.19 seconds with return value 0
Press any key to continue . . . |

```

10. Virtual Memory Using Demand Paging

PSEUDO CODE:

```
Start
Input number of page references
Input page reference string
Input number of frames
Initialize all frames as empty
For each page reference
    Check whether page is already in a frame
    If page is present
        Display Hit
    Else
        Replace page using FIFO
        Count page fault
        Display Fault
    Display contents of all frames
Display total page faults
Stop
```

Code Explanation:

- The program accepts the page reference string and the number of available frames.
- Initially, all memory frames are empty.
- For each page reference, the program checks whether the page is already present in a frame.
- If the page is found, it is a Page Hit, and no replacement is performed.
- If the page is not found, a Page Fault occurs.
- The required page is loaded into memory using the FIFO (First-In, First-Out) page replacement technique.

- After processing each page reference, the program displays the current contents of all memory frames.
- At the end, the program displays the total number of page faults that occurred during execution.

OUTPUT:

```

C:\Users\Anish K\OneDrive\D  ×  +  v
Enter number of page references: 10
Enter page reference string:
1 2 5 6 3 4 5 63 2 4
Enter number of frames: 3

Page    Frames
1        1 - - Fault
2        1 2 - Fault
5        1 2 5 Fault
6        6 2 5 Fault
3        6 3 5 Fault
4        6 3 4 Fault
5        5 3 4 Fault
63       5 63 4 Fault
2        5 63 2 Fault
4        4 63 2 Fault

Total Page Faults = 10

-----
Process exited after 13.35 seconds with return value 0
Press any key to continue . . . |

```

11. Comparison of FIFO, LRU, and Optimal Page Replacement Algorithms

PSEUDO CODE:

```
Start
Input number of page references
Input page reference string
Input number of frames
Simulate FIFO
    Check page hit or fault
    Replace oldest page
    Display frame contents
    Count page faults
Simulate LRU
    Check page hit or fault
    Replace least recently used page
    Display frame contents
    Count page faults
Simulate Optimal
    Check page hit or fault
    Replace page used farthest in future
    Display frame contents
    Count page faults
Display total page faults for FIFO, LRU, and Optimal
Stop
```

Code Explanation:

- The program accepts the page reference string and the number of available frames.
- It simulates three page replacement algorithms: FIFO, LRU, and Optimal.
- In FIFO, the oldest page in memory is replaced whenever a page fault occurs.
- In LRU, the page that has not been used for the longest time is replaced.
- In Optimal, the page whose next use is farthest in the future is replaced, resulting in the minimum possible page faults.

- After every page reference, the program displays the current contents of all memory frames.
- The program counts the number of page faults separately for each algorithm.
- At the end, it displays the total page faults for FIFO, LRU, and Optimal, allowing easy comparison of their performance.

OUTPUT:

```

C:\Users\Anish K\OneDrive\D... × + v
Enter number of page references: 12
Enter page reference string:
7 8 6 2 1 5 3 12 4 5 2 5
Enter number of frames: 3

===== FIFO Page Replacement =====
Page    Frames
7        7 - -
8        7 8 -
6        7 8 6
2        2 8 6
1        2 1 6
5        2 1 5
3        3 1 5
12       3 12 5
4        3 12 4
5        5 12 4
2        5 2 4
5        5 2 4
FIFO Page Faults = 11

===== LRU Page Replacement =====
Page    Frames
7        7 - -
8        7 8 -
6        7 8 6
2        2 8 6
1        2 1 6
5        2 1 5
3        3 1 5
12       3 12 5
4        3 12 4
5        5 12 4
2        5 2 4
5        5 2 4
LRU Page Faults = 11

===== Optimal Page Replacement =====
Page    Frames
7        7 - -
8        7 8 -

```

12. Simulation of Thrashing and Working-Set Management\

PSEUDO CODE:

```
Start
Input number of page references
Input page reference string
Input working-set window size
For each page reference
    Consider the last 'window' references
    Find unique pages in that window
    Display the working set
    Calculate working-set size
    If working-set size is very large
        Display "Thrashing Possible"
Stop
```

Code Explanation :

- The program accepts the page reference sequence and the working-set window size.
- For each page reference, it examines the most recent page references within the specified window.
- It identifies the unique pages in that window to form the working set.
- The program calculates the working-set size, which represents the number of distinct pages currently required by the process.
- The working set and its size are displayed after processing each page reference.
- If the working-set size becomes excessively large compared to the available memory, the program indicates that thrashing may occur.
- Thrashing happens when the system spends more time handling page faults than executing the actual process.

- This simulation helps understand how the working-set model can be used to reduce page faults and improve memory management.

OUTPUT:

```
C:\Users\Anish K\OneDrive\D... x + v
Enter number of page references: 10
Enter page reference string:
1 5 6 3 2 1 4 5 6 5
Enter working-set window size: 3

Working Set Analysis

Reference 1 (1): Working Set = { 1 }   Size = 1
Reference 2 (5): Working Set = { 1 5 }   Size = 2
Reference 3 (6): Working Set = { 1 5 6 }   Size = 3
Reference 4 (3): Working Set = { 5 6 3 }   Size = 3
Reference 5 (2): Working Set = { 6 3 2 }   Size = 3
Reference 6 (1): Working Set = { 3 2 1 }   Size = 3
Reference 7 (4): Working Set = { 2 1 4 }   Size = 3
Reference 8 (5): Working Set = { 1 4 5 }   Size = 3
Reference 9 (6): Working Set = { 4 5 6 }   Size = 3
Reference 10 (5): Working Set = { 5 6 }   Size = 2

-----
Process exited after 24.56 seconds with return value 0
Press any key to continue . . . |
```

13. Simulation of Dynamic Linking

PSEUDO CODE:

```
Start
Input number of symbols
For each symbol
    Input function name
    Input library name
    Input memory address
    Store in symbol table
Input function to search
Search function in symbol table
If function is found
    Display function name
    Display library name
    Display memory address
Else
    Display "Symbol Not Found"
    Display "Dynamic Linking Failed"
Stop
```

Code Explanation:

- The program creates a symbol table to store function names, library names, and memory addresses.
- The user enters the number of symbols and their corresponding details.
- The program accepts the name of the function that needs to be dynamically linked at execution time.
- It searches the symbol table using the strcmp() function.
- If the function is found, the program displays the corresponding library name and memory address.

- If the function is not found, it displays "Symbol Not Found" and indicates that dynamic linking has failed.
- This simulates how an operating system or loader resolves external function references from dynamically linked libraries during program execution.

OUTPUT:

```
C:\Users\Anish K\OneDrive\D  ×  +  v

Symbol 1
Function Name: printf
Library Name: stdio
Memory Address: 1000

Symbol 2
Function Name: scanf
Library Name: stdio
Memory Address: 1050

Symbol 3
Function Name: sqrt
Library Name: math
Memory Address: 2000

Enter function to be linked: sqrt

Symbol Found!
Function : sqrt
Library  : math
Address  : 2000

-----
Process exited after 88.43 seconds with return value 0
Press any key to continue . . . |
```

14. Simulation of a Two-Pass Linker

PSEUDO CODE:

```
Start
Input number of object modules
Pass 1:
    For each module
        Input symbols and addresses
        Store them in External Symbol Table
Display External Symbol Table
Pass 2:
    Input external references
    For each reference
        Search External Symbol Table
        If found
            Display resolved address
        Else
            Display undefined external symbol
Stop
```

Code Explanation:

- The program simulates a two-pass linker used in operating systems.
- In Pass 1, it accepts multiple object modules containing symbols and their addresses.
- All symbols are stored in an External Symbol Table (EST).
- The completed EST is displayed after Pass 1.
- In Pass 2, the program accepts external symbol references.
- Each reference is searched in the External Symbol Table using the strcmp() function.
- If the symbol exists, its linked memory address is displayed.
- If the symbol does not exist, the program reports it as an Undefined External Symbol.

- This demonstrates how a linker combines multiple object modules and resolves external references before generating the executable program.

OUTPUT:

```
C:\Users\Anish K\OneDrive\D  X + v
Enter number of object modules: 2

----- PASS 1 : Construct External Symbol Table -----

Module 1
Enter number of symbols: 2
Symbol Name: A
Address: 100
Symbol Name: B
Address: 200

Module 2
Enter number of symbols: 2
Symbol Name: C
Address: 300
Symbol Name: D
Address: 400

External Symbol Table
-----
Symbol  Address
A       100
B       200
C       300
D       400

----- PASS 2 : Resolve External References -----
Enter number of external references: |

- Compilation Time: 0.34s
```

15. Disk Scheduling Algorithms (FCFS, SSTF, SCAN and C-SCAN)

PSEUDO CODE:

Start
Input number of disk requests
Input disk request sequence
Input initial head position
Input disk size
Perform FCFS scheduling
Display service order
Display total head movement
Perform SSTF scheduling
Display service order
Display total head movement
Perform SCAN scheduling
Display service order
Display total head movement
Perform C-SCAN scheduling
Display service order
Display total head movement
Stop

Code Explanation :

- The program accepts the disk request sequence, initial disk-head position, and disk size.
- FCFS (First Come First Serve) services requests in the order they arrive.
- SSTF (Shortest Seek Time First) always services the request closest to the current head position, reducing seek time.
- SCAN (Elevator Algorithm) moves the disk head in one direction, servicing all requests, reaches the end of the disk, and then reverses direction.
- C-SCAN (Circular SCAN) services requests in one direction only. After reaching the last cylinder, it jumps back to the beginning and continues servicing requests.

- For each algorithm, the program displays the order in which disk requests are serviced.
- It calculates the head movement for each request using the absolute difference between consecutive head positions.
- Finally, it displays the total head movement for each scheduling algorithm, allowing performance comparison.

OUTPUT:

```

C:\Users\Anish K\OneDrive\D  ×  +  v
Enter number of disk requests: 8
Enter disk requests:
125
566 45 68
452
458
56
562
Enter initial head position: 56
Enter disk size: 200

===== FCFS =====
Order: 56 -> 125 -> 566 -> 45 -> 68 -> 452 -> 458 -> 56 -> 562
Total Head Movement = 2352

===== SSTF =====
Order: 56 -> 56 -> 45 -> 68 -> 125 -> 452 -> 458 -> 562 -> 566
Total Head Movement = 532

===== SCAN =====
Order: 56 -> 56 -> 68 -> 125 -> 452 -> 458 -> 562 -> 566 -> 45
Total Head Movement = 1031

===== C-SCAN =====
Order: 56 -> 56 -> 68 -> 125 -> 452 -> 458 -> 562 -> 566 -> 45
Total Head Movement = 1121

-----
Process exited after 31.92 seconds with return value 0
Press any key to continue . . . |

```